OS/161 Assignment 2 Report

Abu Bakar Siddique
Roll: 47

July 8, 2026

1 Implemented Work

- Implemented kernel synchronization primitives: locks and condition variables.
- Implemented process-related system calls: `fork`, `getpid`, `waitpid`, `_exit`, and `execv`.
- Added argument passing so user programs started from the kernel menu receive correct `argc/argv`.
- Implemented basic file system calls: `open`, `read`, `write`, `close`, and `remove`.
- Added file creation with `O_CREAT` and append behavior with `O_APPEND`.

2 Changed Files

- `kern/thread/synch.c`: lock and CV logic.
- `kern/include/proc.h`: PID and file handle structures.
- `kern/proc/proc.c`: PID table, wait support, file table setup/copy/cleanup.
- `kern/syscall/proc_syscalls.c`: process syscalls.
- `kern/syscall/file_syscalls.c`: file syscalls.
- `kern/arch/mips/syscall/syscall.c`: syscall dispatch.
- `kern/syscall/runprogram.c`: first-process argument passing.
- `kern/startup/menu.c`: forwards menu arguments.

3 Code Evidence and Explanation

3.1 System Call Interface

Syscall Prototypes
This screenshot shows the new kernel function declarations in `kern/include/syscall.h`. Functions such as `sys_fork`, `sys_waitpid`, `sys_open`, `sys_read`, and `sys_write` are declared here so the syscall dispatcher can call their implementations. This file acts like the common interface between the low-level trap handler and the syscall source files.
`screenshots/syscall-prototypes.png`

OS 161 Setup > Asst2 > source > os161-1.99 > kern > include > **syscall.h**

```
1  int sys_write(int fdesc,userptr_t ubuf,unsigned int nbytes,int *retval);
2  int sys_open(userptr_t filename, int flags, mode_t mode, int *retval);
3  int sys_close(int fd);
4  int sys_read(int fd, userptr_t ubuf, size_t buflen, int *retval);
5  int sys_remove(userptr_t pathname);
6  void sys__exit(int exitcode);
7  int sys_getpid(pid_t *retval);
8  int sys_waitpid(pid_t pid, userptr_t status, int options, pid_t *retval);
9  int sys_fork(struct trapframe *tf, pid_t *retval);
10  int sys_execv(userptr_t program, userptr_t args);
```

Created with SnippetShot

Syscall Dispatcher

The dispatcher reads the syscall number from the trapframe and uses a `switch` statement to call the correct kernel function. For example, `SYS_open` calls `sys_open`, `SYS_fork` calls `sys_fork`, and return values are placed back into the trapframe registers. This is the main entry point for user programs requesting kernel services.
`screenshots/syscall-dispatcher.png`

1 Setup > Asst2 > source > os161-1.99 > kern > arch > mips > syscall > sys

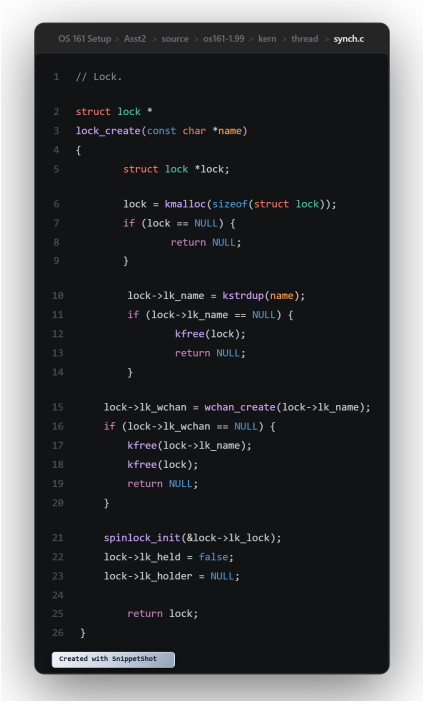
```
1      case SYS_write:
2        err = sys_write((int)tf->tf_a0,
3                (userptr_t)tf->tf_a1,
4                (int)tf->tf_a2,
5                (int *)(&retval));
6        break;
7    #if OPT_A2
8      case SYS_open:
9        err = sys_open((userptr_t)tf->tf_a0,
10               (int)tf->tf_a1,
11               (mode_t)tf->tf_a2,
12               (int *)(&retval));
13       break;
14     case SYS_close:
15       err = sys_close((int)tf->tf_a0);
16       break;
17     case SYS_read:
18       err = sys_read((int)tf->tf_a0,
19               (userptr_t)tf->tf_a1,
20               (size_t)tf->tf_a2,
21               (int *)(&retval));
22       break;
23     case SYS_remove:
24       err = sys_remove((userptr_t)tf->tf_a0);
25       break;
```

Created with SnippetShot

3.2 Synchronization

Lock Create

`lock_create` allocates memory for a lock, copies the lock name, creates the wait channel, initializes the internal spinlock, and sets the initial owner state to free. The wait channel is used when a thread must sleep, while the spinlock protects the lock's internal fields.
`screenshots/lock_create.png`



Lock Acquire and Release

`lock_acquire` first protects the lock state using the internal spinlock. If the lock is already held, the current thread sleeps on the wait channel until it is woken. `lock_release` clears the holder, marks the lock free, and wakes one sleeping thread so another thread can enter the critical section.

`screenshots/lock_acquire+release.png`

Condition Variables

`cv_wait` is used when a thread cannot continue until some condition becomes true. It releases the caller's lock before sleeping and reacquires it after wakeup, preventing deadlock. `cv_signal` wakes one waiting thread, and `cv_broadcast` wakes all threads waiting on the condition variable.

`screenshots/cv_wait+signal+broadcast.png`

3.3 Process System Calls

getpid and waitpid

`sys_getpid` simply returns the PID stored in the current process structure. `sys_waitpid` validates that the requested PID belongs to a child process, sleeps until that child exits, then returns the encoded exit status to the parent through the user pointer. This gives the parent a controlled way to collect child termination information.
`screenshots/sys-gerpid+waitpid.png`

fork

`sys_fork` creates a new process by copying the parent's trapframe and address space. It also copies references to open file handles so the child inherits the parent's open files. After `thread_fork`, the parent returns the child PID, while the child enters user mode with return value 0.
`screenshots/sys-fork.png`

runprogram Arguments

`runprogram_args` loads the executable, creates the user address space, and copies argument strings onto the new user stack. It then builds the `argv` pointer array and passes `argc` and `argv` to `enter_new_process`. This is why menu commands with extra words can be received by user programs.

`screenshots/run-program_args.png`

3.4 File System Calls

File Handle

The file handle structure stores the vnode, current offset, open flags, reference count, and a lock. The vnode points to the actual file object in the VFS layer. The offset tracks where the next read/write happens, and the refcount allows parent and child processes to safely share the same open file after `fork`.

`screenshots/filehandle.png`

61 Setup > Asst2 > source > os161-1.99 > kern > include > pı

```
#if OPT_A2
struct filehandle {
    struct vnode *vn;
    off_t offset;
    int flags;
    int refcount;
    struct lock *lock;
};
```

Created with SnippetShot

Process File Table

Each process owns a file descriptor table named `p_filetable[OPEN_MAX]`. A user file descriptor is just an integer index into this table. When a file is opened, the kernel finds a free slot, stores the file handle there, and returns the slot number to the user program.

`screenshots/filehandle2.png`

OS 161 Setup > Asst2 > source > os161-1.99 > kern > include > proc.h

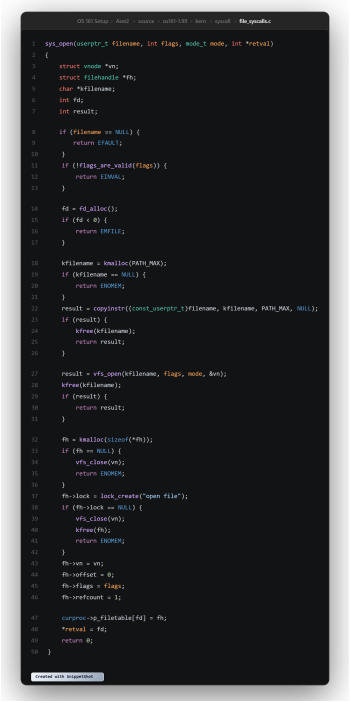
```
#if OPT_A2
    pid_t p_pid;            /* Unique process id, 0 for kproc */
    bool p_counted;         /* True after adding to proc_count */
    struct filehandle *p_filetable[OPEN_MAX];
#endif /* OPT_A2 */
};
```

Created with SnippetShot

open and create

`sys_open` copies the filename from user memory using safe copy functions, checks the open flags, and calls `vfs_open`. After a vnode is returned, it creates a file handle and stores it in the process file table. `O_CREAT` is handled through the VFS open flags so a missing file can be created.

`screenshots/sys-open.png`

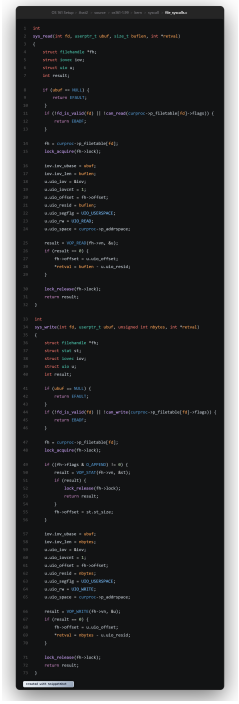


```
sys_open(userptr_t filename, int flags, mode_t mode, int *retval)
{
    struct vnode *vn;
    struct filehandle *fh;
    char *kfilename;
    int fd;
    int result;

    if (filename == NULL) {
        return EFAULT;
    }
    if (!flags_are_valid(flags)) {
        return EINVAL;
    }

    fd = fd_alloc();
    if (fd < 0) {
        return EMFILE;
    }

    kfilename = kmalloc(PATH_MAX);
    if (kfilename == NULL) {
        return ENOMEM;
    }
    result = copyinstr((const_userptr_t)filename, kfilename, PATH_MAX, NULL);
    if (result) {
        kfree(kfilename);
        return result;
    }

    result = vfs_open(kfilename, flags, mode, &vn);
    kfree(kfilename);
    if (result) {
        return result;
    }

    fh = kmalloc(sizeof(*fh));
    if (fh == NULL) {
        vfs_close(vn);
        return ENOMEM;
    }
    fh->lock = lock_create("open file");
    if (fh->lock == NULL) {
        vfs_close(vn);
        kfree(fh);
        return ENOMEM;
    }
    fh->vn = vn;
    fh->offset = 0;
    fh->flags = flags;
    fh->refcount = 1;

    curproc->p_filetable[fd] = fh;
    *retval = fd;
    return 0;
}
```

read, write, and append

`sys_read` and `sys_write` use `uio` and `iovec` objects because OS/161 file I/O must safely transfer bytes between user memory and the kernel/VFS layer. The file handle lock protects the offset while I/O is happening. For `O_APPEND`, `sys_write` first moves the offset to the file size, then writes at the end.

`screenshots/sys-read+write.png`



close and remove

`sys_close` clears the descriptor entry in the process file table and decreases the file handle reference count. When no process uses the handle anymore, the vnode is closed and memory is freed. `sys_remove` copies the pathname from user space and calls `vfs_remove` to delete the file from the mounted filesystem.

`screenshots/sys-close+remove.png`

4 Test Commands

Build/run from Docker:

```
cd /root/cs350-os161
bash ./build-and-run-kernel.sh
```

Important OS/161 menu commands:

```
sy2
sy3
p /testbin/palin
p /testbin/argtest hello os161
mount sfs lhd0
p /testbin/filetest lhd0:testfile
```

5 Output Evidence and Meaning

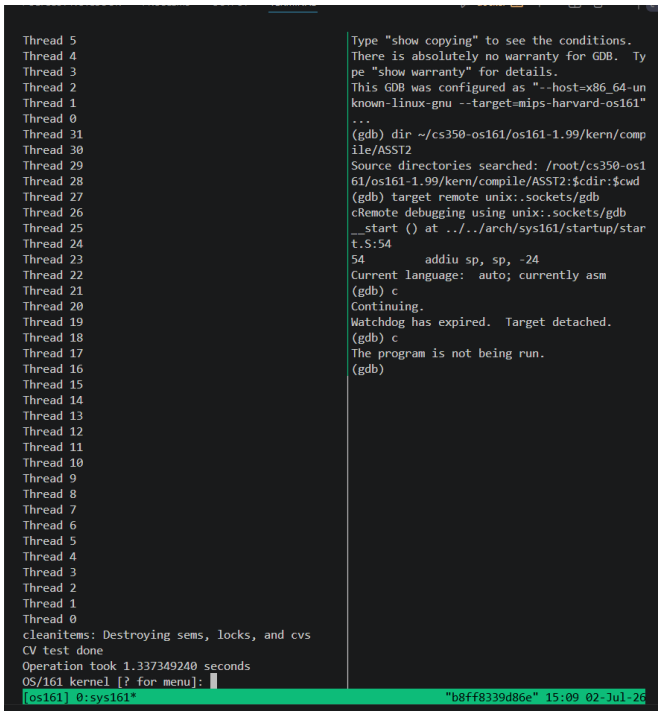
5.1 Lock Test

Lock Test Output
The `sy2` test checks whether locks can be created, acquired, released, and destroyed without corrupting shared state. The final line `Lock test done.` means the lock implementation completed the test normally.
`screenshots/asst2-02-sy2-lock.png`

```
cpu0: MIPS r3000
OS/161 kernel [? for menu]: sy2
Starting lock test...
cleanitems: Destroying sems, locks, and cvs
Lock test done.
Operation took 0.315240160 seconds
OS/161 kernel [? for menu]:
[os161] 0:sys161*
```

5.2 Condition Variable Test

CV Test Output
The `sy3` test starts 32 threads and prints them in reverse order several times. This output means sleeping, waking, signaling, and broadcasting with condition variables are working. The final `CV test done` confirms the test finished.
`screenshots/asst2-03-sy3-cv.png`



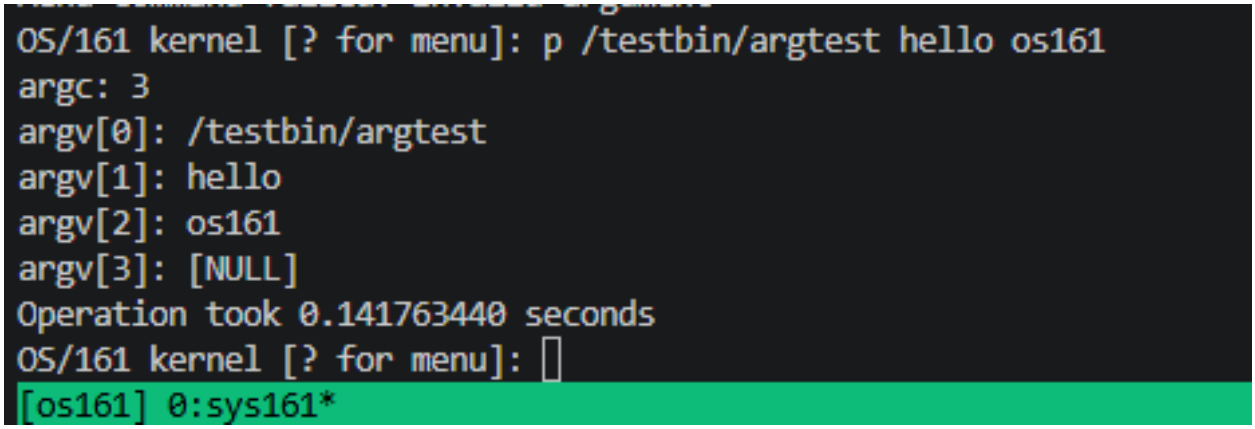
5.3 User Program Execution

Palindrome Program Output
This screenshot shows that `/testbin/palin` runs in user mode. The program prints a long palindrome test and finishes with `IS a palindrome`, proving that user program loading and basic console output work.
`screenshots/asst2-04-palin.png`

5.4 Argument Passing

Argument Passing Output

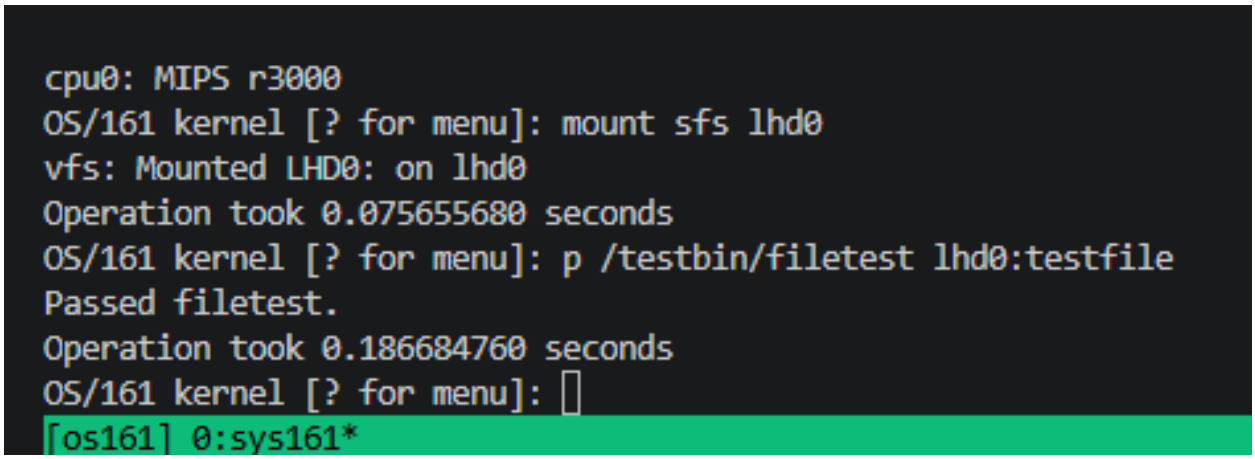
The command `p /testbin/argtest hello os161` prints `argc: 3` and shows the expected values in `argv`. This proves the kernel correctly copies command-line arguments onto the new process stack.
`screenshots/asst2-05-argtest.png`



5.5 File System Calls

File Syscall Output

The filesystem is mounted using `mount sfs lhd0`, then `filetest` runs on `lhd0:testfile`. The message `Passed filetest.` means create/open, read, write, close, append behavior, and remove are functioning for the tested case.
`screenshots/asst2-06-filetest.png`


cpu0: MIPS r3000
OS/161 kernel [? for menu]: mount sfs lhd0
vfs: Mounted LHD0: on lhd0
Operation took 0.075655680 seconds
OS/161 kernel [? for menu]: p /testbin/filetest lhd0:testfile
Passed filetest.
Operation took 0.186684760 seconds
OS/161 kernel [? for menu]:
[os161] 0:sys161*